Lab S09 · Mensajería con RabbitMQ

Amy Veraza · Sistemas Distribuidos 2027-1 · Grupo 2

Práctica del 17 de octubre de 2026. Fecha de entrega: 24 de octubre de 2026, antes de las 07:00.

Mexi Banco: que el aviso no tumbe la transferencia. Base: rama oficial 05-mensajeria-base. Entrega individual: código y evidencias obtenidas en esta máquina.

Entorno: Docker Desktop, servicios ejecutados con Java 21, PostgreSQL 17 y RabbitMQ 4. Gateway: http://localhost:18090; panel: http://localhost:15672. Se utilizó una réplica de cuenta. Los archivos de evidencias conservan respuestas HTTP, estados de las colas y logs reales; las capturas del panel se tomaron con navegador.

1. El problema de la llamada síncrona

En la versión base apagué notificación y transferí 50 de Ana a Beto. Los saldos pasaron de 1000/500 a 950/550; el dinero llegó, pero la llamada HTTP de aviso falló antes de marcar COMPLETADA. La fila quedó ABONADA y la petición devolvió un error. La causa fue acoplar la terminación de la saga a la disponibilidad de un aviso que podía esperar.

Evidencia: 01-problema-http.txt


```
{
  "http": 500,
  "respuesta": {
    "timestamp": "2026-10-23T05:17:24.462+00:00",
    "status": 500,
    "error": "Internal Server Error",
    "path": "/transferencias"
  }
}
```


Evidencia: 01-problema-saldos.txt


```
[
  {
    "http": 200,
    "respuesta": {
      "id": 1,
      "clabe": "002180000000000001",
      "titular": "Ana",
      "saldo": 950.0
    }
  },
  {
    "http": 200,
    "respuesta": {
      "id": 2,
      "clabe": "002180000000000002",
      "titular": "Beto",
      "saldo": 550.0
    }
  }
]
```


Evidencia: 01-problema-abonada.txt


```
{
  "http": 200,
  "respuesta": [
    {
      "id": 1,
      "claveOrigen": "002180000000000001",
      "claveDestino": "002180000000000002",
      "monto": 50.0,
```

```
        "fecha": "2026-10-23T05:17:23.885186Z",
        "estado": "ABONADA",
        "motivo": null,
        "saldoOrigenTrasCargo": 950.0,
        "intentosDeCompensacion": 0,
        "actualizada": "2026-10-23T05:17:24.253841Z"
      }
    ]
  }
```




2. Publicar el hecho de negocio

ConfiguracionRabbit declara transferencias como exchange topic durable y configura RabbitTemplate con Jackson2JsonMessageConverter. La saga guarda COMPLETADA y publica TransferenciaRealizada con transferencia.realizada. Si compensa el cargo, guarda COMPENSADA y publica TransferenciaRevertida con transferencia.revertida, también cuando termina un reintento programado. El aviso ya no requiere una petición HTTP a notificación.

El evento contiene eventoId, tipo, transferenciaId, clabeOrigen, clabeDestino, monto, saldoOrigen, saldoDestino y fecha. La identidad es un UUID derivado de la transferencia y el tipo: volver a publicar el mismo hecho conserva eventoId. Para la revertida se informa el saldo del origen después de la devolución y saldoDestino es null, pues no se abonó al destino.

El publicador usa mensajes persistentes, publisher-confirm-type: correlated, publisher-returns y mandatory. Espera el confirm y revisa mensajes retornados. Un confirm positivo indica que RabbitMQ aceptó el mensaje; por sí solo no demuestra que hubiera una cola enlazada ni que el consumidor haya guardado el aviso. Mandatory permite detectar una publicación sin ruta. Ninguna de estas confirmaciones vuelve atómica la escritura en PostgreSQL y la publicación en RabbitMQ.

```
        // Identidad estable aun si el mismo hecho se publica de nuevo tras un reinicio.
        UUID id = UUID.nameUUIDFromBytes((t.clave(tipo)).getBytes(StandardCharsets.UTF_8));
        EventoTransferencia evento = new EventoTransferencia(id, tipo, t.getId(),
t.getClaveOrigen(),
            t.getClaveDestino(), t.getMonto(), saldoOrigen, saldoDestino, t.getActualizada());
        String key = "TransferenciaRealizada".equals(tipo) ? "transferencia.realizada" :
"transferencia.revertida";
        CorrelationData confirmacion = new CorrelationData(id.toString());
        try {
            rabbit.convertAndSend("transferencias", key, evento, mensaje -> {
                mensaje.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT);
                mensaje.getMessageProperties().setMessageId(id.toString());
                return mensaje;
            }, confirmacion);
            var resultado = confirmacion.getFuture().get(5, TimeUnit.SECONDS);
            if (!resultado.isAck() || confirmacion.getReturned() != null) {
                throw new IllegalStateException("Sin confirmacion o sin cola enlazada");
            }
            log.info("Evento {} confirmado: {}", id, key);
        } catch (Exception fallo) {
            if (fallo instanceof InterruptedException) Thread.currentThread().interrupt();
            // Sin outbox: no romper una transferencia que ya movio el dinero. Se documenta la
perdida.
```

3. Desacople en tiempo

Arranqué primero notificación para declarar su cola durable y el binding transferencia.*. Luego la apagué e hice tres transferencias, de 11, 12 y 13. Las tres devolvieron 201 y COMPLETADA. Con el consumidor detenido, la cola conservó exactamente tres mensajes Ready. Al encenderlo, guardó dos avisos por evento y la cola volvió a cero.

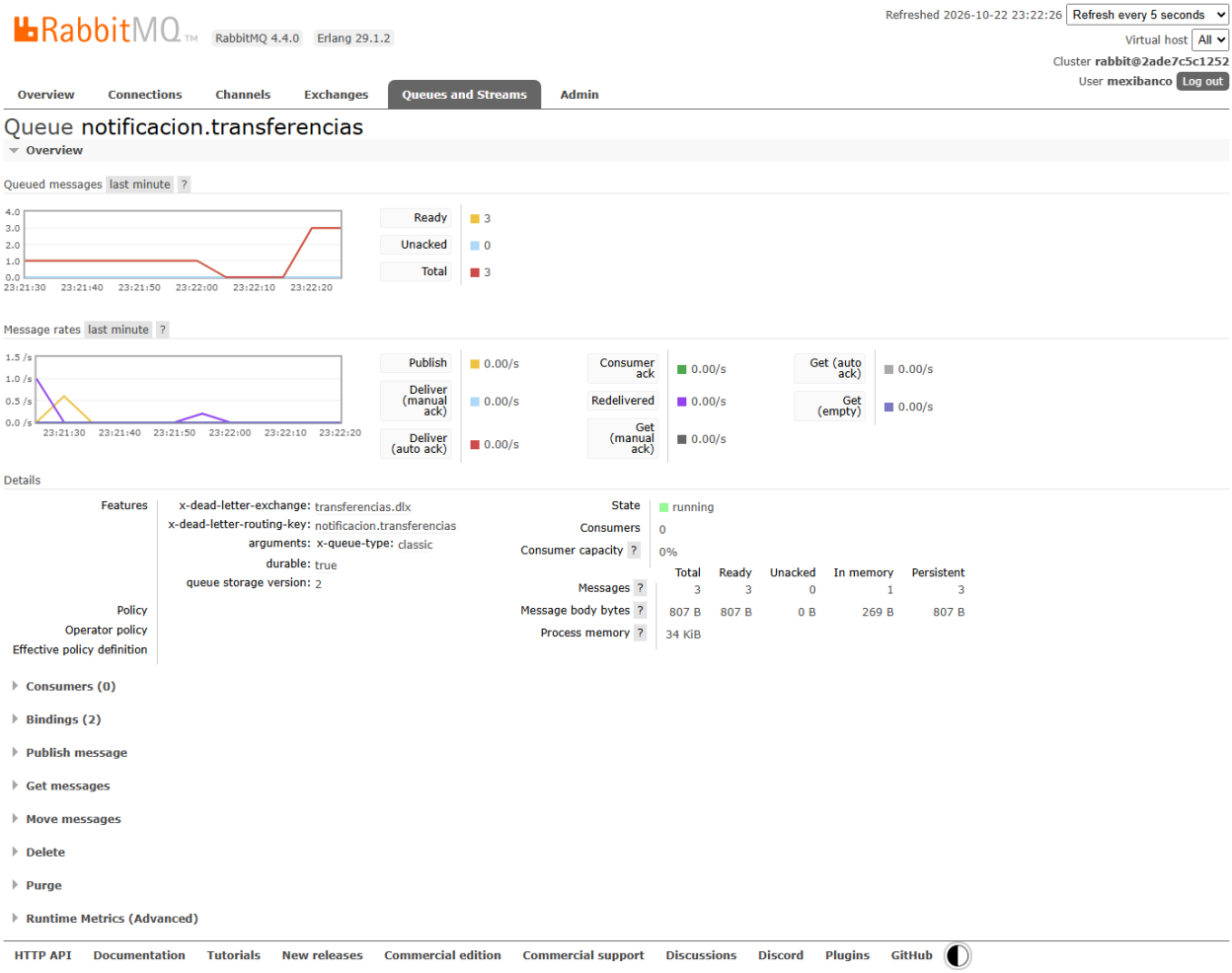
Evidencia: 02-tres-transferencias.txt


```
[
  {
    "http": 201,
    "respuesta": {
      "id": 5,
      "claveOrigen": "0021800000000000001",
      "claveDestino": "0021800000000000002",
      "monto": 11,
      "fecha": "2026-10-23T05:22:11.719093495Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 903.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:11.999522782Z"
    }
  },
  {
    "http": 201,
    "respuesta": {
      "id": 6,
      "claveOrigen": "0021800000000000001",
      "claveDestino": "0021800000000000002",
      "monto": 12,
      "fecha": "2026-10-23T05:22:12.092358288Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 891.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:12.187748312Z"
    }
  },
  {
    "http": 201,
    "respuesta": {
      "id": 7,
      "claveOrigen": "0021800000000000001",
      "claveDestino": "0021800000000000002",
      "monto": 13,
      "fecha": "2026-10-23T05:22:12.262859100Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 878.0,
```

```
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:12.363347198Z"
    }
  }
]
```

Evidencia: 02-cola-exacta.txt

```
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 1       0
notificacion.transferencias     3       0
```



Tres transferencias completadas con el consumidor apagado

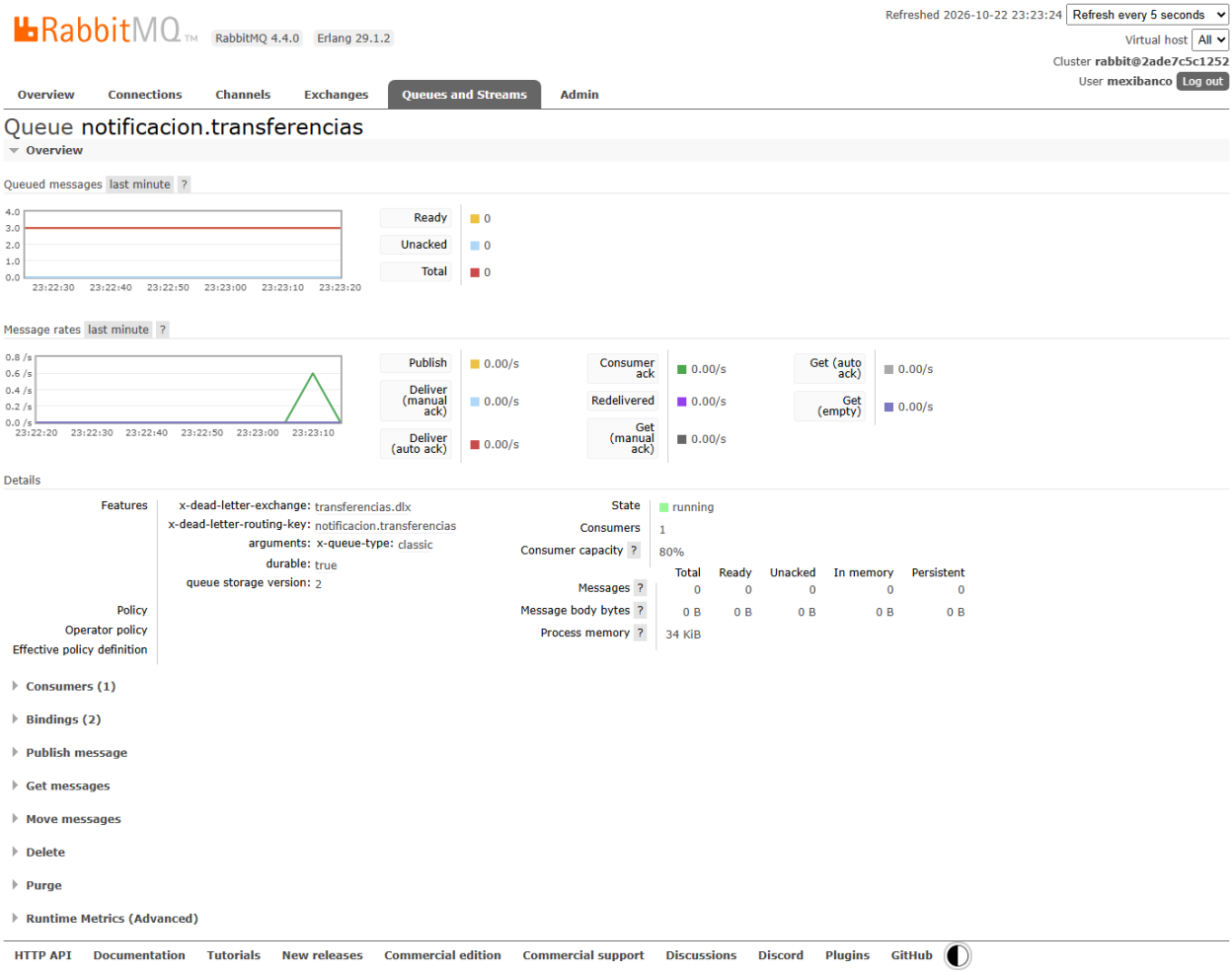
Vista de archivos de evidencia reales; contenido íntegro en evidencias/

02-tres-transferencias.txt


```
[
  {
    "http": 201,
    "respuesta": {
      "id": 5,
      "claveOrigen": "00218000000000001",
      "claveDestino": "00218000000000002",
      "monto": 11,
      "fecha": "2026-10-23T05:22:11.719093495Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 903.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:11.999522782Z"
    }
  },
  {
    "http": 201,
    "respuesta": {
      "id": 6,
      "claveOrigen": "00218000000000001",
      "claveDestino": "00218000000000002",
      "monto": 12,
      "fecha": "2026-10-23T05:22:12.092358288Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 891.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:12.187748312Z"
    }
  },
  {
    "http": 201,
    "respuesta": {
      "id": 7,
      "claveOrigen": "00218000000000001",
      "claveDestino": "00218000000000002",
      "monto": 13,
      "fecha": "2026-10-23T05:22:12.262859100Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 878.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:22:12.363347198Z"
    }
  }
]
```


Evidencia: 02-cola-vacia.txt

```
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 1       0
notificacion.transferencias     0       0
```



Evidencia: 02-bandejas.txt

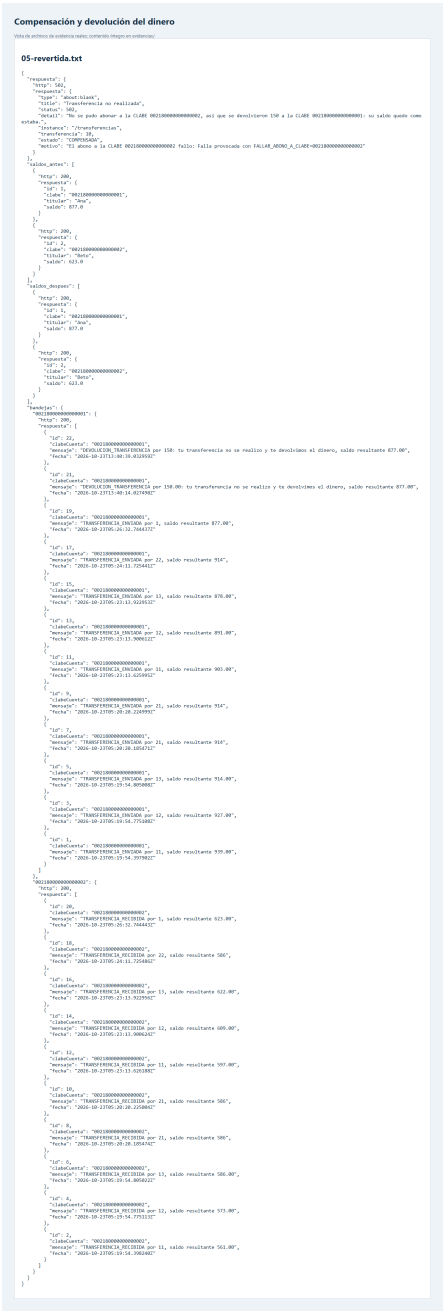

```
{
  "002180000000000001": {
    "http": 503,
    "respuesta": {
      "timestamp": "2026-10-23T05:23:18.363+00:00",
      "path": "/api/notificaciones",
      "status": 503,
      "error": "Service Unavailable",
      "requestId": "fd38ea5c-232"
    }
  },
  "002180000000000002": {
    "http": 503,
    "respuesta": {
      "timestamp": "2026-10-23T05:23:18.379+00:00",
      "path": "/api/notificaciones",
      "status": 503,
      "error": "Service Unavailable",
      "requestId": "68d5c085-233"
    }
  }
}
```


También provoqué el fallo de abono hacia Beto con FALLAR_ABONO_A_CLABE. La saga devolvió el cargo: respondió 502 y COMPENSADA, los saldos quedaron como antes del intento y Ana recibió un aviso DEVOLUCION_TRANSFERENCIA por 150.

Evidencia: 05-revertida.txt


```
{
  "respuesta": {
    "http": 502,
    "respuesta": {
```

```
      "type": "about:blank",
      "title": "Transferencia no realizada",
      "status": 502,
      "detail": "No se pudo abonar a la CLABE 002180000000000002, asi que se devolvieron 150 a la CLABE
002180000000000001: su saldo quedo como estaba.",
      "instance": "/transferencias",
      "transferencia": 10,
      "estado": "COMPENSADA",
      "motivo": "El abono a la CLABE 002180000000000002 fallo: Falla provocada con
FALLAR_ABONO_A_CLABE=002180000000000002"
    }
  },
  "saldos_antes": [
    {
      "http": 200,
      "respuesta": {
        "id": 1,
        "clabe": "002180000000000001",
        "titular": "Ana",
        "saldo": 877.0
      }
    },
    {
      "http": 200,
      "respuesta": {
        "id": 2,
        "clabe": "002180000000000002",
        "titular": "Beto",
        "saldo": 623.0
      }
    }
  ],
  "saldos_despues": [
    {
      "http": 200,
      "respuesta": {
        "id": 1,
        "clabe": "002180000000000001",
        "titular": "Ana",
        "saldo": 877.0
      }
    },
    {
      "http": 200,
[Salida completa en evidencias/]
```



Compensación y devolución del dinero

Vista de archivos de evidencia reales; contenido íntegro en evidencias/

05-revertida.txt


```
{
  "respuesta": {
    "http": 502,
    "respuesta": {
      "type": "about:blank",
      "title": "Transferencia no realizada",
      "status": 502,
      "detail": "No se pudo abonar a la CLABE 002180000000000002, así que se devolvieron 150 a la CLABE 002180000000000001: su saldo queda como estaba.",
      "instance": "/transferencias",
      "transferenciaId": 1,
      "estado": "COMPENSADA",
      "motivo": "El abono a la CLABE 002180000000000002 falló: falla provocada con FALLAR_ABONO_A_CLABE=002180000000000002"
    }
  },
  "saldos_antes": [
    {
      "http": 200,
      "respuesta": {
        "id": 1,
        "clabe": "002180000000000001",
        "titular": "Ana",
        "saldo": 877.0
      }
    },
    {
      "http": 200,
      "respuesta": {
        "id": 2,
        "clabe": "002180000000000002",
        "titular": "Beto",
        "saldo": 623.0
      }
    }
  ],
  "saldos_despues": [
    {
      "http": 200,
      "respuesta": {
        "id": 1,
        "clabe": "002180000000000001",
        "titular": "Ana",
        "saldo": 877.0
      }
    },
    {
      "http": 200,
      "respuesta": {
        "id": 2,
        "clabe": "002180000000000002",
        "titular": "Beto",
        "saldo": 623.0
      }
    }
  ],
  "bandejas": {
    "002180000000000001": {
      "http": 200,
      "respuesta": [
        {
          "id": 22,
          "claveCuenta": "002180000000000001",
          "mensaje": "DEVOLUCION_TRANSFERENCIA por 150: tu transferencia no se realizo y te devolvimos el dinero, saldo resultante 877.00",
          "fecha": "2026-10-23T11:40:19.461893Z"
        },
        {
          "id": 21,
          "claveCuenta": "002180000000000001",
          "mensaje": "DEVOLUCION_TRANSFERENCIA por 150.00: tu transferencia no se realizo y te devolvimos el dinero, saldo resultante 877.00",
          "fecha": "2026-10-23T13:40:14.027498Z"
        },
        {
          "id": 19,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 1, saldo resultante 877.00",
          "fecha": "2026-10-23T05:26:32.744127Z"
        },
        {
          "id": 17,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 22, saldo resultante 914",
          "fecha": "2026-10-23T05:24:11.725441Z"
        },
        {
          "id": 15,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 13, saldo resultante 878.00",
          "fecha": "2026-10-23T05:23:11.622953Z"
        },
        {
          "id": 11,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 12, saldo resultante 891.00",
          "fecha": "2026-10-23T05:23:11.900612Z"
        },
        {
          "id": 11,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 31, saldo resultante 903.00",
          "fecha": "2026-10-23T05:23:11.625895Z"
        },
        {
          "id": 9,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 21, saldo resultante 914",
          "fecha": "2026-10-23T05:20:10.234995Z"
        },
        {
          "id": 7,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 31, saldo resultante 914",
          "fecha": "2026-10-23T05:20:10.105471Z"
        },
        {
          "id": 5,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 13, saldo resultante 914.00",
          "fecha": "2026-10-23T05:19:54.865008Z"
        },
        {
          "id": 3,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 12, saldo resultante 927.00",
          "fecha": "2026-10-23T05:19:54.775488Z"
        },
        {
          "id": 1,
          "claveCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 21, saldo resultante 939.00",
          "fecha": "2026-10-23T05:19:54.397902Z"
        }
      ]
    },
    "002180000000000002": {
      "http": 200,
      "respuesta": [
        {
          "id": 20,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 1, saldo resultante 623.00",
          "fecha": "2026-10-23T05:26:32.744443Z"
        },
        {
          "id": 18,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 22, saldo resultante 586",
          "fecha": "2026-10-23T05:24:11.725488Z"
        },
        {
          "id": 16,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 13, saldo resultante 622.00",
          "fecha": "2026-10-23T05:23:11.922956Z"
        },
        {
          "id": 14,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 12, saldo resultante 609.00",
          "fecha": "2026-10-23T05:23:11.900624Z"
        },
        {
          "id": 12,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 11, saldo resultante 597.00",
          "fecha": "2026-10-23T05:23:11.625388Z"
        },
        {
          "id": 10,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 21, saldo resultante 586",
          "fecha": "2026-10-23T05:20:10.225004Z"
        },
        {
          "id": 8,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 21, saldo resultante 586",
          "fecha": "2026-10-23T05:20:10.105474Z"
        },
        {
          "id": 6,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 13, saldo resultante 586.00",
          "fecha": "2026-10-23T05:19:54.865022Z"
        },
        {
          "id": 4,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 12, saldo resultante 573.00",
          "fecha": "2026-10-23T05:19:54.775115Z"
        },
        {
          "id": 2,
          "claveCuenta": "002180000000000002",
          "mensaje": "TRANSFERENCIA_RECIBIDA por 11, saldo resultante 561.00",
          "fecha": "2026-10-23T05:19:54.396268Z"
        }
      ]
    }
  }
}
```


4. Duplicado: antes y después

Para observar el problema usé una etapa experimental sin deduplicación y publiqué dos veces el mismo JSON con el mismo eventoId. Aparecieron dos avisos nuevos por cuenta. Esta publicación manual reproduce la entrega duplicada de un evento; no representa dos movimientos de dinero.

Evidencia: 03-duplicado-antes.txt


```
{
  "evento": {
    "eventoId": "0c4d9095-e37b-4ecf-bccb-c821344a240a",
    "tipo": "TransferenciaRealizada",
    "transferenciaId": 9001,
    "clabeOrigen": "0021800000000000001",
    "clabeDestino": "0021800000000000002",
    "monto": 21,
    "saldoOrigen": 914,
    "saldoDestino": 586,
    "fecha": "2026-10-22T12:00:00Z"
  },
  "avisos_nuevos_por_cuenta": {
```

```
    "002180000000000001": 2,
    "002180000000000002": 2
  },
  "bandejas": {
    "002180000000000001": {
      "http": 200,
      "respuesta": [
        {
          "id": 9,
          "clabeCuenta": "002180000000000001",
[Salida completa en evidencias/]
```

La solución final usa evento_procesado con evento_id como llave primaria. Dentro del mismo método público @Transactional se ejecuta INSERT ... ON CONFLICT DO NOTHING y se guardan los avisos. Si la inserción devuelve cero, el evento ya estaba procesado y no se repiten avisos. La restricción también protege frente a consumidores concurrentes; no hay una consulta previa vulnerable a carreras. Si guardar falla, el rollback revierte tanto la marca como los avisos.

El listener recibe el resultado después del commit del servicio y sólo entonces hace basicAck. Repetí el experimento con otro evento duplicado: cada cuenta obtuvo exactamente un aviso nuevo. El segundo procesamiento devuelve cero avisos y también se confirma.

```
    @Transactional
    public int avisar(EventoTransferencia e) {
        e.validar();
        if (procesados.marcarSiEsNuevo(e.eventoId(), e.tipo(), e.transferenciaId()) == 0) return 0;
        if ("TransferenciaRealizada".equals(e.tipo())) {
            avisos.saveAll(List.of(
                new Notificacion(e.clabeOrigen(), "TRANSFERENCIA_ENVIADA por " + e.monto() + ",
saldo resultante " + e.saldoOrigen()),
                new Notificacion(e.clabeDestino(), "TRANSFERENCIA_RECIBIDA por " + e.monto() + ",
saldo resultante " + e.saldoDestino())));
            return 2;
        }
        avisos.save(new Notificacion(e.clabeOrigen(), "DEVOLUCION_TRANSFERENCIA por " + e.monto()
            + ": tu transferencia no se realizo y te devolvimos el dinero, saldo resultante " +
e.saldoOrigen()));
        return 1;
    }
}
```

Evidencia: 03-duplicado-despues.txt

```
{
  "evento": {
    "eventoId": "2af753ec-41e0-448f-9f1d-ddbe4d21c81f",
    "tipo": "TransferenciaRealizada",
    "transferenciaId": 9002,
    "clabeOrigen": "002180000000000001",
    "clabeDestino": "002180000000000002",
    "monto": 22,
    "saldoOrigen": 914,
    "saldoDestino": 586,
    "fecha": "2026-10-22T12:00:00Z"
  },
  "avisos_nuevos_por_cuenta": {
    "002180000000000001": 1,
    "002180000000000002": 1
  },
  "bandejas": {
    "002180000000000001": {
      "http": 200,
      "respuesta": [
        {
          "id": 17,
          "clabeCuenta": "002180000000000001",
[Salida completa en evidencias/]
```

Evidencia: 04-logs-despues.txt

```
notificacion-1  | ( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
notificacion-1  |  \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
notificacion-1  |   '  |____| .__|_| |_|_| |_\__, | / / / /
notificacion-1  |  =========|_|==============|___/=/_/_/_/
```

```
notificacion-1  |
notificacion-1  |  :: Spring Boot ::                (v3.3.4)
notificacion-1  |
notificacion-1  | 2026-10-23T05:22:39.366Z  INFO 1 --- [notificacion] [           main]
mx.mexibanco.NotificacionApplication     : Starting NotificacionApplication v0.9.0 using Java 21.0.12.1 with
PID 1 (/app/app.jar started by root in /app)
notificacion-1  | 2026-10-23T05:22:39.380Z  INFO 1 --- [notificacion] [           main]
mx.mexibanco.NotificacionApplication     : No active profile set, falling back to 1 default profile: "default"
notificacion-1  | 2026-10-23T05:22:43.934Z  INFO 1 --- [notificacion] [           main]
.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
notificacion-1  | 2026-10-23T05:22:44.454Z  INFO 1 --- [notificacion] [           main]
.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 479 ms. Found 2 JPA
repository interfaces.
notificacion-1  | 2026-10-23T05:22:45.966Z  INFO 1 --- [notificacion] [           main]
o.s.cloud.context.scope.GenericScope     : BeanFactory id=fc99041f-1da4-34e6-a325-d22ec806a41c
notificacion-1  | 2026-10-23T05:22:47.255Z  WARN 1 --- [notificacion] [           main]
trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.cloud.client.loadbalancer.LoadBalancerAut
oConfiguration$DeferringLoadBalancerInterceptorConfig' of type [org.springframework.cloud.client.loadbalancer.
LoadBalancerAutoConfiguration$DeferringLoadBalancerInterceptorConfig] is not eligible for getting processed by
all BeanPostProcessors (for example: not eligible for auto-proxying). The currently created BeanPostProcessor
[lbRestClientPostProcessor] is declared through a non-static factory method on that class; consider declaring
it as static instead.
notificacion-1  | 2026-10-23T05:22:47.265Z  WARN 1 --- [notificacion] [           main]
trationDelegate$BeanPostProcessorChecker : Bean 'deferringLoadBalancerInterceptor' of type
[org.springframework.cloud.client.loadbalancer.DeferringLoadBalancerInterceptor] is not eligible for getting
processed by all BeanPostProcessors (for example: not eligible for auto-proxying). Is this bean getting
eagerly injected into a currently created BeanPostProcessor [lbRestClientPostProcessor]? Check the
corresponding BeanPostProcessor declaration and its dependencies.
notificacion-1  | 2026-10-23T05:22:48.319Z  INFO 1 --- [notificacion] [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8080 (http)
notificacion-1  | 2026-10-23T05:22:48.378Z  INFO 1 --- [notificacion] [           main]
o.apache.catalina.core.StandardService   : Starting service [Tomcat]
notificacion-1  | 2026-10-23T05:22:48.379Z  INFO 1 --- [notificacion] [           main]
o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.30]
notificacion-1  | 2026-10-23T05:22:48.494Z  INFO 1 --- [notificacion] [           main]
o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
notificacion-1  | 2026-10-23T05:22:48.497Z  INFO 1 --- [notificacion] [           main]
w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 8951 ms
notificacion-1  | 2026-10-23T05:22:51.447Z  INFO 1 --- [notificacion] [           main]
o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
notificacion-1  | 2026-10-23T05:22:52.187Z  INFO 1 --- [notificacion] [           main] org.hibernate.Version
[Salida completa en evidencias/]
```

Duplicado: efecto antes y después de la idempotencia

Vista de archivos de evidencia reales; contenido íntegro en evidencias/

03-duplicado-antes.txt


```
{
  "evento": {
    "eventoId": "0c4d9095-e37b-4ecf-bccb-c821344a240a",
    "tipo": "TransferenciaRealizada",
    "transferenciaId": 9001,
    "clabeOrigen": "002180000000000001",
    "clabeDestino": "002180000000000002",
    "monto": 21,
    "saldoOrigen": 914,
    "saldoDestino": 586,
    "fecha": "2026-10-22T12:00:00Z"
  },
  "avisos_nuevos_por_cuenta": {
    "002180000000000001": 2,
    "002180000000000002": 2
  },
  ...
}
```


03-duplicado-despues.txt

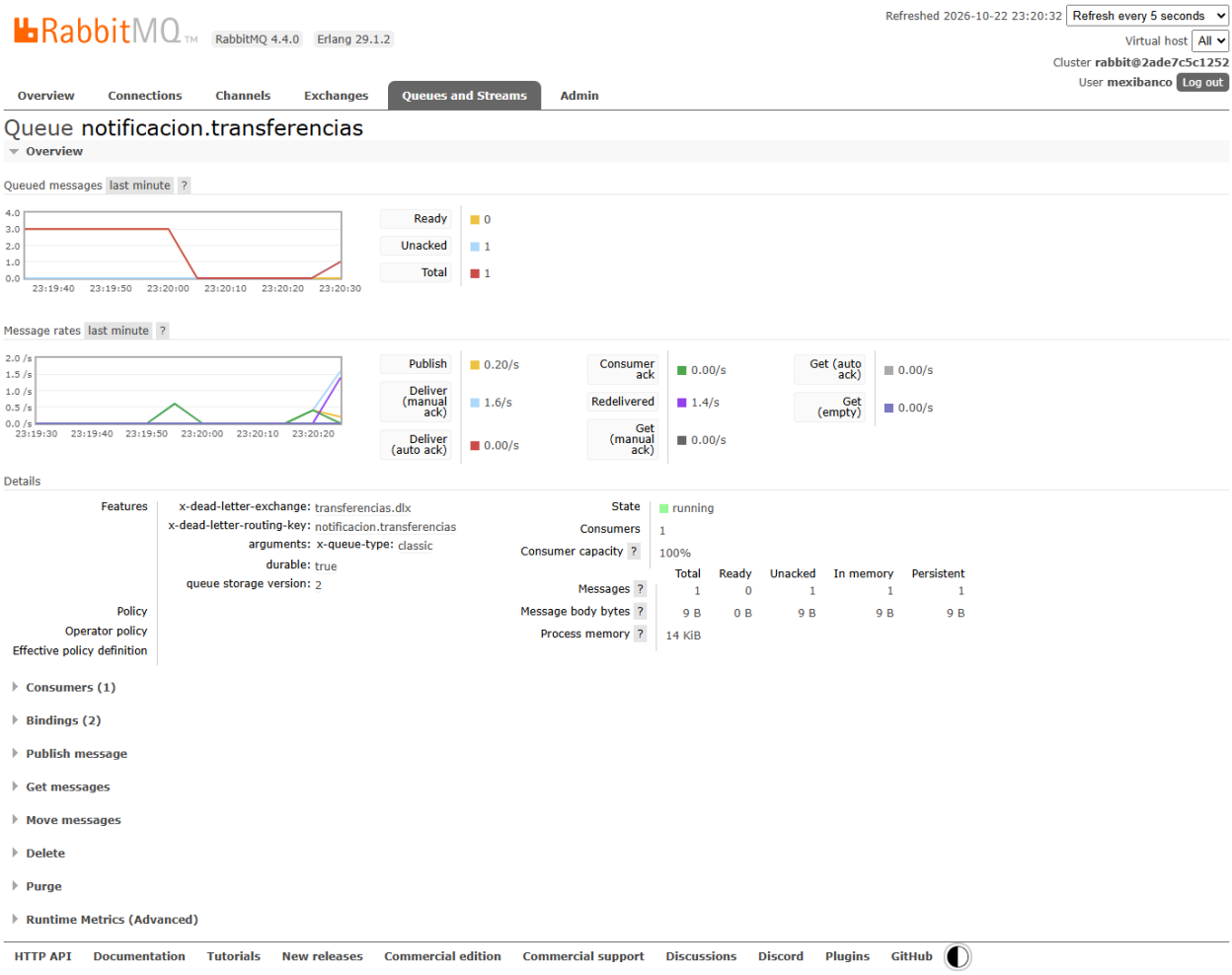

```
{
  "evento": {
    "eventoId": "2af753ec-41e0-448f-9f1d-ddbe4d21c81f",
    "tipo": "TransferenciaRealizada",
    "transferenciaId": 9002,
    "clabeOrigen": "002180000000000001",
    "clabeDestino": "002180000000000002",
    "monto": 22,
    "saldoOrigen": 914,
    "saldoDestino": 586,
    "fecha": "2026-10-22T12:00:00Z"
  },
  "avisos_nuevos_por_cuenta": {
    "002180000000000001": 1,
    "002180000000000002": 1
  },
  ...
}
```


5. Mensaje envenenado y DLQ

Publiqué el cuerpo incompleto {"hola": con transferencia.realizada. En la etapa experimental el consumidor lo rechazaba con requeue=true: se entregaba otra vez y los logs repetían el error de JSON. Este reintento no podía corregir el contenido.

Evidencia: 04-veneno-antes.txt


```
{
  "cola": {
    "name": "notificacion.transferencias",
    "durable": true,
    "arguments": {
      "x-dead-letter-exchange": "transferencias.dlx",
      "x-dead-letter-routing-key": "notificacion.transferencias",
      "x-queue-type": "classic"
    },
    "messages": 1,
    "messages_ready": 0,
    "messages_unacknowledged": 1,
    "consumers": 1
  },
  "dlq": {
    "name": "notificacion.transferencias.dlq",
    "durable": true,
    "arguments": {
      "x-queue-type": "classic"
    },
    "messages": 0,
    "messages_ready": 0,
    "messages_unacknowledged": 0,
    "consumers": 0
  }
}
```

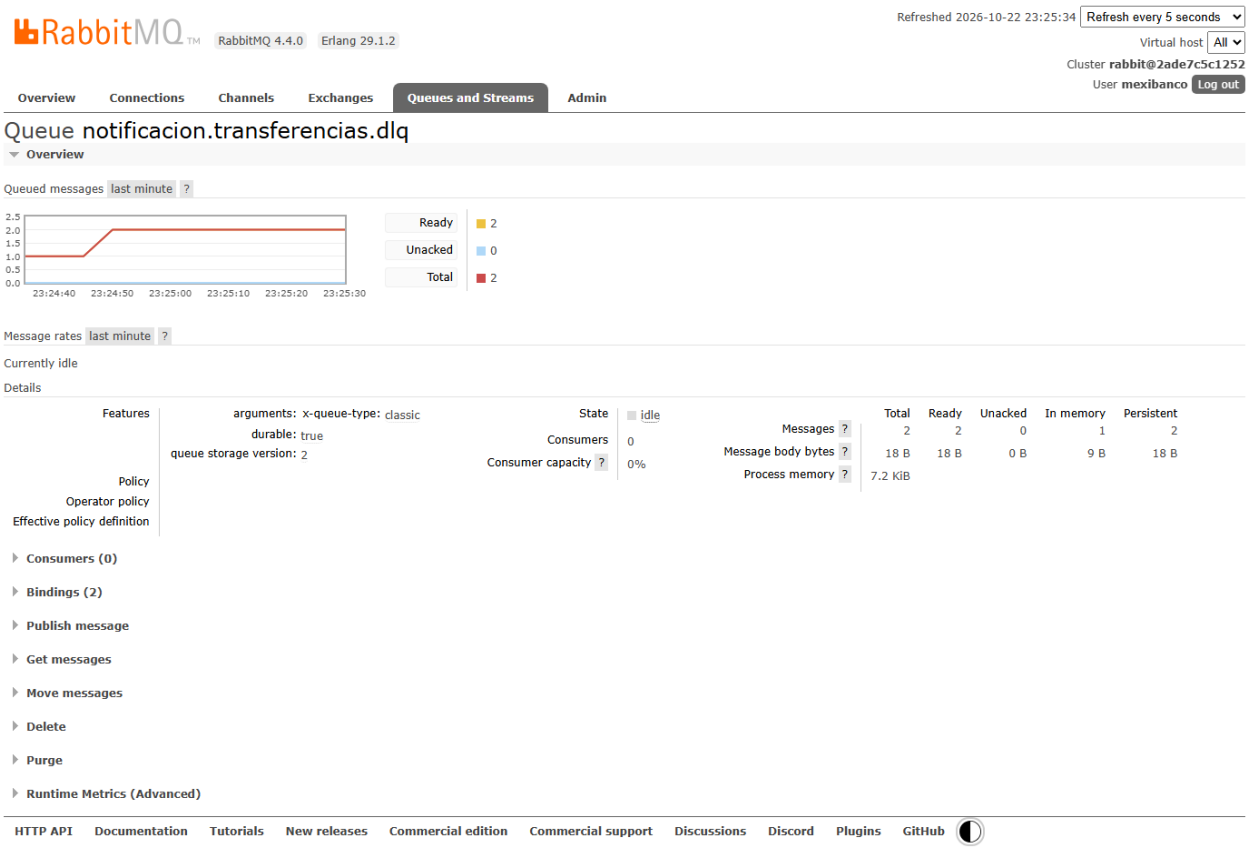


La versión final declara transferencias.dlx como exchange direct durable y notificacion.transferencias.dlq como cola durable. La cola principal tiene x-dead-letter-exchange=transferencias.dlx y x-dead-letter-routing-key=notificacion.transferencias. Cuando falla el parseo o la validación del contrato se ejecuta basicNack con requeue=false: RabbitMQ mueve el mensaje a la DLQ. Un tipo desconocido o un evento sin identidad también se rechazan. Un error de acceso a base se reencola, porque el mensaje puede ser válido y el fallo temporal.

Evidencia: 04-veneno-despues.txt


```
{
  "cola": {
    "name": "notificacion.transferencias",
    "durable": true,
    "arguments": {
      "x-dead-letter-exchange": "transferencias.dlx",
      "x-dead-letter-routing-key": "notificacion.transferencias",
      "x-queue-type": "classic"
    },
    "messages": 0,
    "messages_ready": 0,
    "messages_unacknowledged": 0,
    "consumers": 1
  },
  "dlq": {
    "name": "notificacion.transferencias.dlq",
    "durable": true,
    "arguments": {
      "x-queue-type": "classic"
    },
    "messages": 2,
    "messages_ready": 2,
    "messages_unacknowledged": 0,
    "consumers": 0
```

```
    }
  }
```



Después del rechazo hice una transferencia normal de 1. Respondió 201 y generó los avisos: el mensaje inválido quedó apartado sin impedir el trabajo posterior. No borré la cola para modificarla: la topología con DLX se declaró desde el inicio de las etapas de mensajería; el antes se provocó cambiando solamente el modo de rechazo.

Evidencia: 04-normal-tras-veneno.txt


```
{
  "transferencia": {
    "http": 201,
    "respuesta": {
      "id": 8,
      "claveOrigen": "0021800000000000001",
      "claveDestino": "0021800000000000002",
      "monto": 1,
      "fecha": "2026-10-23T05:26:31.638714186Z",
      "estado": "COMPLETADA",
      "motivo": null,
      "saldoOrigenTrasCargo": 877.0,
      "intentosDeCompensacion": 0,
      "actualizada": "2026-10-23T05:26:32.563398197Z"
    }
  },
  "bandejas": {
    "0021800000000000001": {
      "http": 200,
      "respuesta": [
        {
          "id": 19,
          "clabeCuenta": "0021800000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 1, saldo resultante 877.00",
          "fecha": "2026-10-23T05:26:32.744437Z"
        },
        {
          "id": 17,
```

```
          "clabeCuenta": "002180000000000001",
          "mensaje": "TRANSFERENCIA_ENVIADA por 22, saldo resultante 914",
[Salida completa en evidencias/]
```

6. Preguntas de cierre

1. Si RabbitMQ está caído, mi versión conserva la transferencia COMPLETADA y registra un error de publicación. Como no implementé Outbox, no existe un reenvío automático garantizado: el aviso puede perderse. El confirm sólo sirve si se logra publicar y recibir una respuesta del broker; no puede recuperar el evento si el proceso muere después del commit o si no había conexión. Para cerrar esa ventana tendría que guardar estado final y evento pendiente en una misma transacción y enviarlo desde un relay.

2. El ack va después del commit para que una caída antes de guardar no pierda el aviso: el broker lo vuelve a entregar. Si el proceso cae después del commit y antes del ack, habrá una entrega repetida; la idempotencia evita repetir el efecto. Si confirmara antes de guardar, una caída entre ambas acciones eliminaría el mensaje sin haber guardado el aviso. El orden elegido da entrega al menos una vez y, mediante la transacción y la llave única, un solo efecto local por eventoId.

3. Elección: uso avisos asíncronos mediante una cola durable. Porque: la caída de notificación no debe impedir cerrar una transferencia cuyo dinero ya llegó. Sacrifico: la confirmación inmediata del aviso; el cliente puede ver la transferencia completada antes de que aparezca su notificación, y debo operar colas, duplicados y mensajes fallidos.

7. Validación y límites

Para la devolución se actualizaron cuenta y movimiento a la misma base S09. Se descartó una corrida que reutilizaba imágenes antiguas, se migró la restricción del tipo de movimiento y se restablecieron los saldos de las cuentas de prueba antes de repetir el caso. Las evidencias principales de esta sección corresponden a la repetición con los servicios actualizados.

Las pruebas automatizadas verifican la saga y el contrato del consumidor: 18 pruebas de transferencia y 5 de notificación, sin fallos ni errores. Se compilaron con el JDK disponible generando bytecode Java 21; las aplicaciones de los experimentos corrieron en Java 21 dentro de Docker. Los experimentos adicionales comprobaron el SQL idempotente contra PostgreSQL real y las colas contra RabbitMQ real.

No se implementó el reto opcional Outbox. Continúa existiendo la ventana de doble escritura entre estado final y broker. Los volúmenes datos y mensajes conservan PostgreSQL y RabbitMQ. La carpeta codigo contiene únicamente los dos servicios modificados y el compose; para ejecutar se colocan sobre la base 05-mensajeria-base, que aporta cuenta, movimiento, discovery, gateway y db/init.

Evidencia: estado-contenedores.txt

```
NAME                              IMAGE                             COMMAND                  SERVICE
CREATED              STATUS                          PORTS
mexi-banco-s09-cuenta-1           mexi-banco-s09-cuenta             "java -jar app.jar"      cuenta          About
a minute ago   Up About a minute (healthy)
mexi-banco-s09-db-1               postgres:17-alpine                "docker-entrypoint.s…"   db              8
hours ago          Up 8 hours (healthy)          5432/tcp
mexi-banco-s09-discovery-1        mexi-banco-s09-discovery          "java -jar app.jar"      discovery       8
hours ago          Up 8 hours (healthy)          0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-s09-gateway-1          mexi-banco-s09-gateway            "java -jar app.jar"      gateway         8
hours ago          Up 8 hours (healthy)          0.0.0.0:18090->8080/tcp, [::]:18090->8080/tcp
mexi-banco-s09-movimiento-1       mexi-banco-s09-movimiento         "java -jar app.jar"      movimiento      3
minutes ago        Up 3 minutes (healthy)
mexi-banco-s09-notificacion-1     mexi-banco-s09-notificacion       "java -jar app.jar"      notificacion    8
hours ago          Up 8 hours (healthy)
mexi-banco-s09-rabbitmq-1         rabbitmq:4-management             "docker-entrypoint.s…"   rabbitmq        8
hours ago          Up 8 hours (healthy)          0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp,
0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
```

```
mexi-banco-s09-transferencia-1   mexi-banco-s09-transferencia   "java -jar app.jar"      transferencia   26
[Salida completa en evidencias/]
```